# Web, Windows and Mac

One product. Three ways to open it. No app-store listing required.

This addendum expands the master plan and its Windows-focused sketches to include web and macOS. It describes the proposed release path; a production website and downloadable apps do not exist yet.

## What users would do

| Choice on our website | What happens |
|---|---|
| Open web app | Sign in through a browser. No download needed. |
| Download Windows | Download Elevateo-Mail-Setup.exe, install it, then open it from Start or the taskbar. |
| Download Mac | Download a .dmg, open it and drag Elevateo Mail into Applications. Open it from Applications or the Dock. |

Proposed formats: a Tauri NSIS .exe for Windows and a signed, notarized .dmg for macOS. Add an .msi only if company IT needs it. Offer Apple Silicon and Intel downloads only for architectures we actually test. [1, 2]

## Share the interface and the backend

Reuse the React screens in the browser and in a small Tauri desktop shell. Keep one cloud backend for mail, campaigns, permissions and the 30-per-mailbox daily limit. Desktop-specific code handles notifications, secure credentials, updates and the local AI connection.

The desktop should load packaged interface files. A webpage must not acquire access to native commands simply because it is displayed inside the app. Share code without sharing unrestricted computer access.

## The important AI difference

**Local Codex / Claude integration starts in the desktop app.** A normal browser page cannot directly start an installed CLI. Installing the website as a PWA does not remove that boundary. Users authenticate their own supported CLI; connection status should explain missing installation or sign-in.

The web app can read mail, compose manually and manage campaigns. Already approved, fully prepared campaigns continue in the cloud when the computer is off. New local-AI work waits until the desktop is available. A paired browser helper could be added later, after a separate security review.

Browser notifications depend on browser/OS support and permission. Do not promise closed-browser notification parity before testing. Native desktop apps remain the first target for reliable local integration.

# Release safely, keep costs small

## Direct downloads still need trusted releases

**Mac:** use Apple Developer ID signing and notarization for normal direct distribution. This does not put the app in the App Store. Apple Developer Program membership currently costs US$99 per year; local pricing can differ. Build with a macOS machine or runner and verify installation on a real Mac. [3, 4]

**Windows:** plan publisher signing for public releases. A newly signed app can still trigger SmartScreen warnings, so signing is not a guarantee of a warning-free first launch. Choose a signing service only after checking eligibility and price. [5]

Treat signing, build runners, domains and native test hardware as separate release costs. They are not covered by the under-A$15/month backend hosting goal. The browser version avoids desktop signing costs.

## A small download and update service

Proposed setup: one HTTPS website, static web assets and versioned installer files in Cloudflare R2. Keep source code in the private GitHub repository; customers download installers without needing repository access. Account permissions still protect all mail and campaign data.

Use Tauri signed updates over HTTPS. Update signatures are separate from Windows/Apple code signing. Back up signing keys securely, publish complete release files before update metadata, and defer installation while work is active. Check for updates sparingly to limit background activity. [6]

## Only the release checks that matter

Use container tests for shared UI/backend behaviour. Native packaging needs separate Windows and Mac checks: install, sign in, reconnect a mailbox, find the CLI, show notifications, update without losing data, and uninstall. Test rejected signatures and interrupted updates. Confirm old desktop clients still work with the current backend.

Before building: settle supported OS versions, Mac chip coverage, signing ownership and access to a Mac. Verify CLI sandboxing, credential storage and OAuth return handling on each OS. Existing backend/desktop standards still require approval; this plan does not imply those features are implemented.

## Primary references


[1] Tauri: Windows installers

[2] Tauri: Mac DMG installation

[3] Apple: Developer ID distribution

[4] Apple: membership and annual fee

[5] Tauri: Windows signing and SmartScreen

[6] Tauri: signed desktop updater